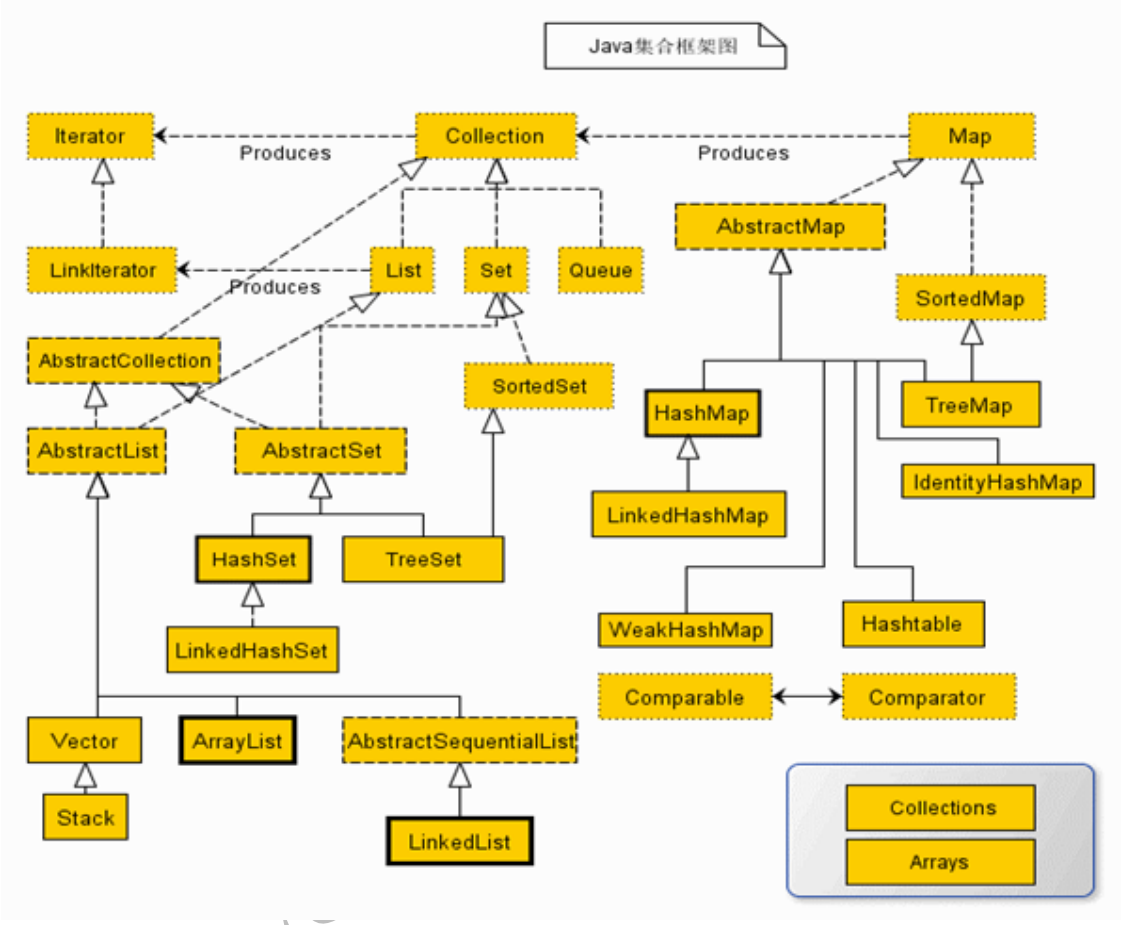
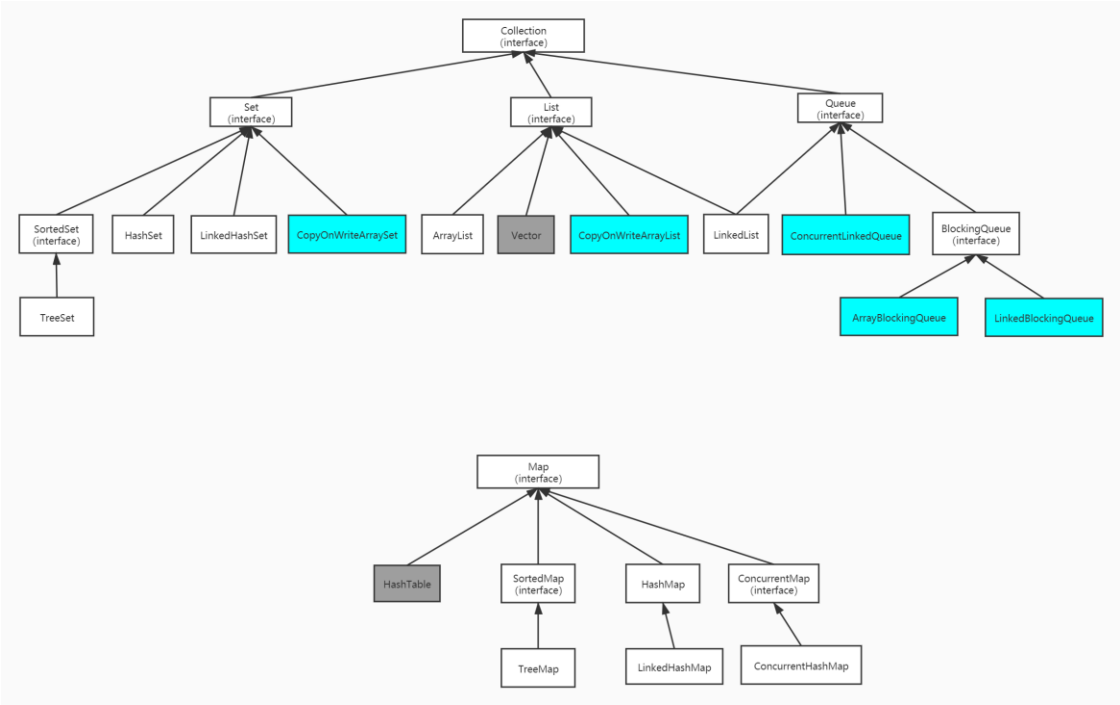
1 HashMap 和 CHM 详解

1.1 集合





1.2 Hash 表

hash 函数->MD5、SHA

通过 hash 函数来计算数据位置的数据结构

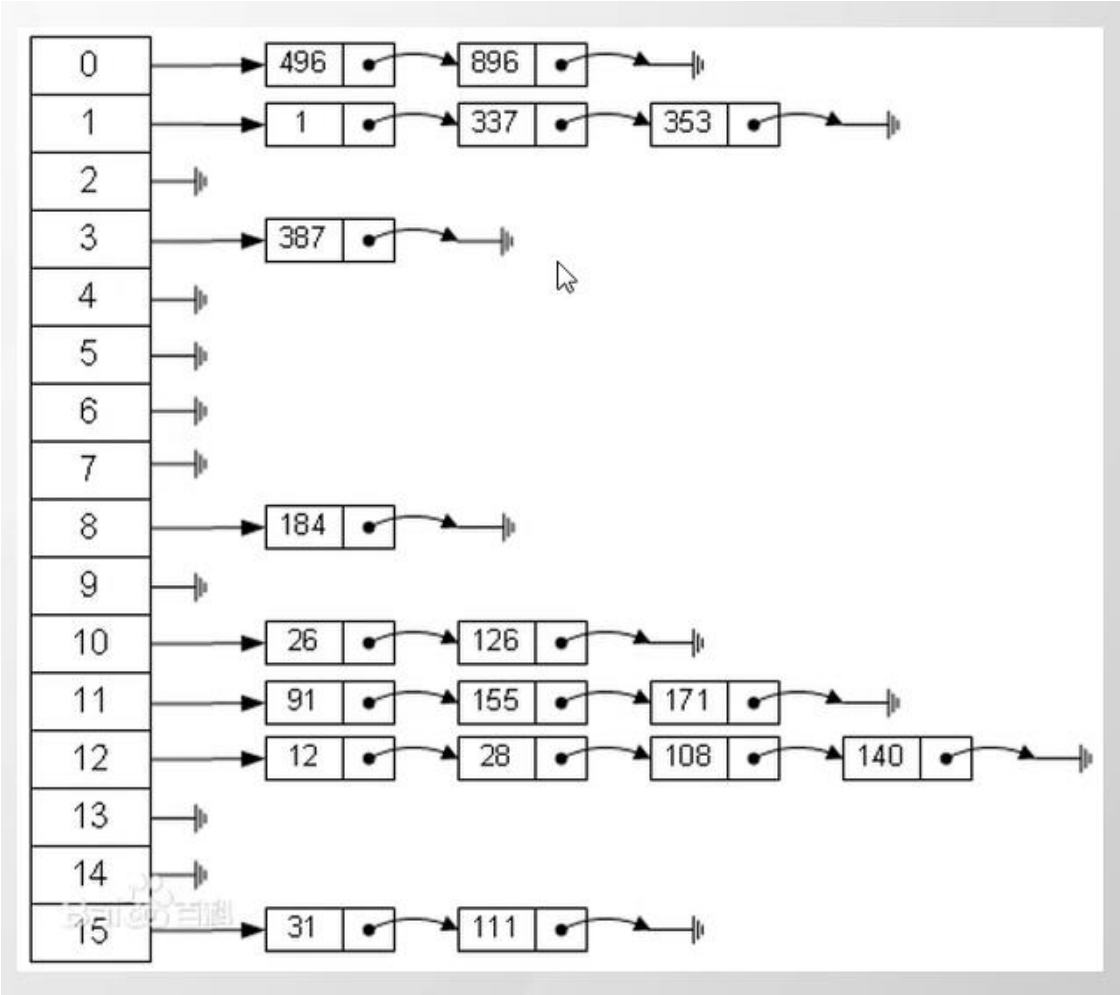
HashMap 通过数组去存储

1.3 Hash 冲突

多个不同的 key 通过 hash 函数运算之后落在同一个数组下标的位置。

1）ThreadLocal 解决方案：线性探索（开发寻址法），i->位置；i+1;i+2

2）HashMap 解决方案：链式地址法，



111110000:496

1111

1110000000:896

1111

3）再 Hash 法（通过多个 hash 函数）->布隆过滤器（位图）

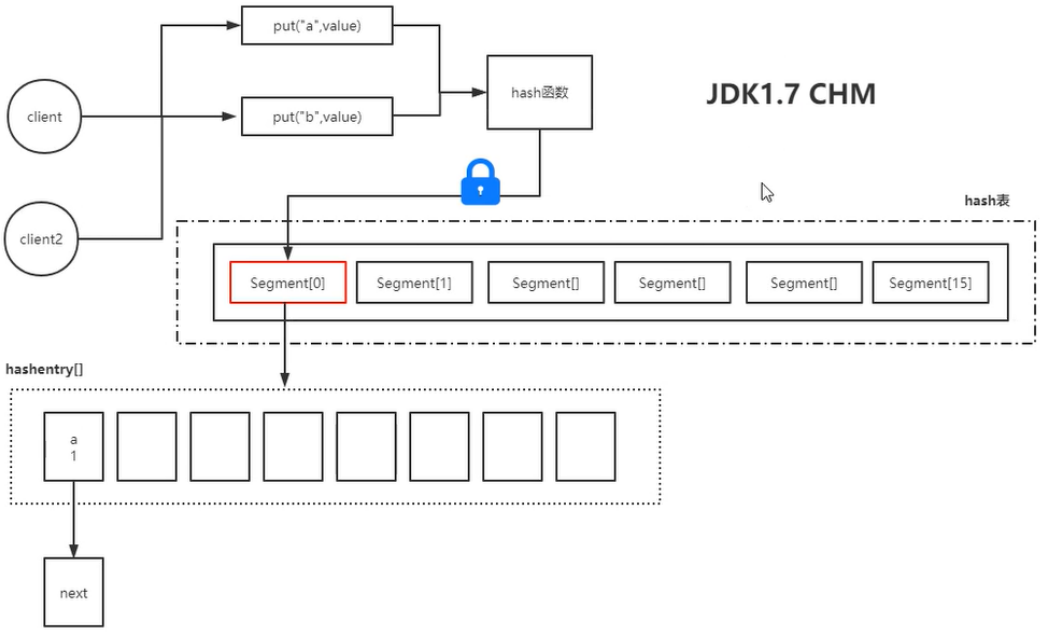
4）建立公共溢出区（基本表和溢出表）

1.4 加锁

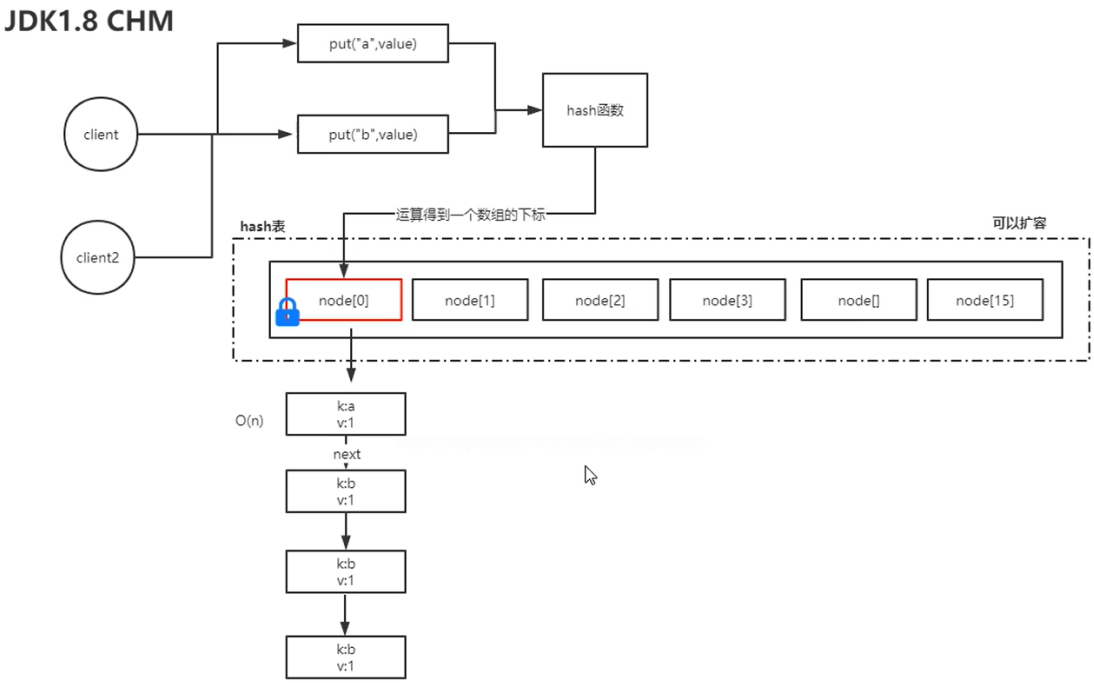
hashtable：是对整个数组加锁，两个线程对同一个 hashtable 进行操作时，只可能有一个线程进行 put。

1.7CHM：对数组中的每个 segment 加锁，不同线程对同一个 CHM 进行操作时，如果 hash 计算没有落在同一个 segment 时，就可以同时操作。每个 segment 里面

存的也是一个 16 个长度的数组，然后数组里面存链表



18CHM：对数组中的每个 node 加锁，不同线程对同一个 CHM 进行操作时，如果 hash 计算没有落在同一个 node 时，就可以同时操作。每个 node 里面存的是存链表或红黑树。



1.5HashMap

1.5.1 从 new 说起

```
HashMap hashMap=new HashMap(5);

public HashMap(int initialCapacity, float loadFactor) {
    if (initialCapacity < 0)
        throw new IllegalArgumentException("Illegal initial capacity: "
 +
                                           initialCapacity);
    if (initialCapacity > MAXIMUM_CAPACITY)
        initialCapacity = MAXIMUM_CAPACITY;
    if (loadFactor <= 0 || Float.isNaN(loadFactor))
        throw new IllegalArgumentException("Illegal load factor: " +
                                           loadFactor);
    this.loadFactor = loadFactor;
    this.threshold = tableSizeFor(initialCapacity);  //代表hashMap 的容
量大于这个的时候会进行resize 操作
}
```

不管传多大的容量，容量都是最靠近设置值的 2 的幂次方

1.5.2 右移左移补充：

<<表示左移移，不分正负数，低位补 0；

注：以下数据类型默认为 byte-8 位

左移时不管正负，低位补 0

正数：**r = 20 << 2**

20 的二进制补码：**0001 0100**

向左移动两位后：**0101 0000**

结果：**r = 80**

r = 20 << 3

20 的二进制补码：**0001 0100**

向左移动两位后：**1010 0000**

原码：1110 0000

结果：**r = -96**

电泡出品 必属精品

负数：**r = -20 << 2**

-20 的二进制原码：**1001 0100**

-20 的二进制反码：**1110 1011**

-20 的二进制补码：**1110 1100**

左移两位后的补码：**1011 0000**

反码：**1100 1111**

原码：**1101 0000**

结果：**r = -80**

>>表示右移，如果该数为正，则高位补 0，若为负数，则高位补 1；

注：以下数据类型默认为 byte-8 位

正数：**r = 20 >> 2**

20 的二进制补码：**0001 0100**

向右移动两位后：**0000 0101**

结果：**r = 5**

负数：**r = -20 >> 2**

-20 的二进制原码：**1001 0100**

-20 的二进制反码：**1110 1011**

-20 的二进制补码：**1110 1100**

右移两位后的补码：**1111 1011**

反码：**1000 0100**

原码：**1000 0101**

结果：**r = -5**

>>>表示无符号右移，也叫逻辑右移，即若该数为正，则高位补 0，而若该数为负数，则右移后高位同样补 0

正数：**r = 20 >>> 2**

的结果与 r = 20 >> 2 相同；

负数：**r = -20 >>> 2**

陈泡出品 必属精品

注：以下数据类型默认为 int 32 位

-20:源码：**10000000 00000000 00000000 00010100**

反码：**11111111 11111111 11111111 11101011**

补码：**11111111 11111111 11111111 11101100**

右移：**00111111 11111111 11111111 11111011**

结果：**r = 1073741819**

-1 源码： 10000000 00000000 00000000 00000001

反码： 11111111 11111111 11111111 11111110

补码： 11111111 11111111 11111111 11111111

01111111 11111111 11111111 11111111

11111111 11111111 11111111 11111111

```
//保证
static final int tableSizeFor(int cap) {
    int n = cap - 1;  //减1.防止传入的刚好是2的幂次方的时候，比如传入4，不减1，会得到7;
    n |= n >>> 1;  //右移一位，然后|运算，能保证高2位是1，是4
                                        n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;   //类推到最大的2的32-1，也就是32个1！！因为java中int4字节，也就是32位，达到int的最大值
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;  //再加1.
}
```

举例 传入的 cap 是 14

```
n=cap-1 ;  13    二进制位 0000 1101

n |= n >>> 1;     n>>>1 为 0000 0110    与 n|运算     得到 00001111
n |= n >>> 2;     n>>>2 为 0000 0011    与 n|运算     得到 00001111
n |= n >>> 4;     n>>>4 为 0000 0000    与 n|运算     得到 00001111
最终结果就是 00010000
```

1.5.3 扰动函数

```
static final int hash(Object key) {
    int h;
```

咕泡出品 必属精品

```
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

右移 16 位与本来的异或；能使高 16 位与低 16 位都参与 hash 运算！！减少 hash 冲突

比如

```
1110 1000  >>>4   0000 1110
0000 1110
---------------
1110 0110
```

1.5.4 继续 put 方法

```
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}

final V putVal(int hash, K key, V value, boolean onlyIfAbsent,
               boolean evict) {
    Node<K,V>[] tab; Node<K,V> p; int n, i;
    //判断table 是否为空
    if ((tab = table) == null || (n = tab.length) == 0)
        //初始化
        n = (tab = resize()).length;
    if ((p = tab[i = (n - 1) & hash]) == null)   //不存在hash 冲突
        tab[i] = newNode(hash, key, value, null);
    else {    //hash 冲突
        Node<K,V> e; K k;
        if (p.hash == hash &&  //hash 值相等
            ((k = p.key) == key || (key != null && key.equals(k))))  //
key 也相等
            e = p;  //直接将已有的给到e
        else if (p instanceof TreeNode) //是红黑树
            e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, val
ue);
        else {   //插入链表
            for (int binCount = 0; ; ++binCount) {   //一直遍历，没有条件
                if ((e = p.next) == null) {   //到了链表尾部
                    p.next = newNode(hash, key, value, null); //添加到链
表尾部
                    if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1s
t   //当链表长度达到8，转变为红黑树
                        treeifyBin(tab, hash);
                    break;
                }
                if (e.hash == hash &&
```

咕泡出品 必属精品

```
                    ((k = e.key) == key || (key != null && key.equals
(k))))   //hash 跟key 都一样，链表里面覆盖
                    break;
                p = e;
            }
        }
        if (e != null) { // existing mapping for key   //有需要覆盖的节
点，覆盖
            V oldValue = e.value;
            if (!onlyIfAbsent || oldValue == null)
                e.value = value;
            afterNodeAccess(e);
            return oldValue;
        }
    }
    ++modCount;
    if (++size > threshold)
        resize();
    afterNodeInsertion(evict);
    return null;
}
```

1.5.5 扩容（初始化）

```
final Node<K,V>[] resize() {
    Node<K,V>[] oldTab = table;
    int oldCap = (oldTab == null) ? 0 : oldTab.length;  //得到原来table
的长度.假如是4
    int oldThr = threshold;  //得到原来的扩容临界值   4*0.75=3
    int newCap, newThr = 0;  //新的table 的长度与扩容临界值
    if (oldCap > 0) {  //如果table 有值，代表不是初始化
        if (oldCap >= MAXIMUM_CAPACITY) {   //判断原来的table 长度是不是
到了最大值，如果到了最大值，不再扩容，直接返回原来的数组
            threshold = Integer.MAX_VALUE;
            return oldTab;
        }
        else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY &&   //newCa
p 为oldCap 的2 倍，并且原来的长度>=16，临界值为原有临界值的2 倍，这时newCap=
8
                oldCap >= DEFAULT_INITIAL_CAPACITY)
            newThr = oldThr << 1; // double threshold,结果是跟newCap*0.
75 一样的，必须大于等于16，不然4 的扩容是3，*2 就会有问题
    }
    else if (oldThr > 0) // initial capacity was placed in threshold  /
/当没有数据的时候，初始化容量就为阈（yu）值
```

哈尼出品 必属精品

```
        newCap = oldThr;
    else {               // zero initial threshold signifies using defa
ults   //默认值
        newCap = DEFAULT_INITIAL_CAPACITY;
        newThr = (int)(DEFAULT_LOAD_FACTOR * DEFAULT_INITIAL_CAPACITY);
    }
    if (newThr == 0) {  //当原来table的长度小于16的时候
        float ft = (float)newCap * loadFactor;   //newCap为原来长度的2
倍*0.75      也就是8*0.75=6
        newThr = (newCap < MAXIMUM_CAPACITY && ft < (float)MAXIMUM_CAPA
CITY ?
                  (int)ft : Integer.MAX_VALUE);    //新的扩容临界值为6
    }
    threshold = newThr;  //扩容值变成了6
    @SuppressWarnings({"rawtypes","unchecked"})   //不用再编译完成后出现
警告信息，忽略"rawtypes","unchecked"
        Node<K,V>[] newTab = (Node<K,V>[])new Node[newCap];  // 新的数组
长度为8
    table = newTab;     //table赋值为newTab
    if (oldTab != null) {  //进行数据迁移
        for (int j = 0; j < oldCap; ++j) {
            Node<K,V> e;
            if ((e = oldTab[j]) != null) {
                oldTab[j] = null;   //便于GC
                if (e.next == null)  //没有next，代表不是链表，就是数组
                    newTab[e.hash & (newCap - 1)] = e;   //hash取模放到
数组位置
                else if (e instanceof TreeNode)  //如果是红黑树，按红黑树
转移
                    ((TreeNode<K,V>)e).split(this, newTab, j, oldCap);
                else { // preserve order   //链表，链表转移
                    //低位链表，也就是当hash取模小于old的时候
                    Node<K,V> loHead = null, loTail = null;
                    //高位链表，也就是当hash取模大于等于old的时候
                    Node<K,V> hiHead = null, hiTail = null;
                    Node<K,V> next;
                    do {
                        next = e.next;
                        if ((e.hash & oldCap) == 0) {
                            if (loTail == null)
                                loHead = e;
                            else
                                loTail.next = e;
                            loTail = e;
                        }
                        else {
                            if (hiTail == null)
                                hiHead = e;
```

咕泡出品 必属精品

```
                        else
                            hiTail.next = e;
                        hiTail = e;
                    }
                } while ((e = next) != null);
                //低位链表放在原有位置
                if (loTail != null) {
                    loTail.next = null;
                    newTab[j] = loHead;
                }
                //如果超过了原来old，放在old 的长度+j
                if (hiTail != null) {
                    hiTail.next = null;
                    newTab[j + oldCap] = hiHead;
                }
            }
        }
    }
}
    return newTab;
}
```

1.5.6 链表转红黑树

```
final void treeifyBin(Node<K,V>[] tab, int hash) {
    int n, index; Node<K,V> e;
    if (tab == null || (n = tab.length) < MIN_TREEIFY_CAPACITY)   //数组不到64，也不会转红黑树
        resize();  //扩容
    else if ((e = tab[index = (n - 1) & hash]) != null) {
        TreeNode<K,V> hd = null, tl = null;   // 定义首、尾节点
        do {
            TreeNode<K,V> p = replacementTreeNode(e, null);  //将该节点转换为 树节点
            if (tl == null)
                hd = p;
            else {
                p.prev = tl;
                tl.next = p;
            }
            tl = p;
        } while ((e = e.next) != null);  // 继续遍历链表

          // 到目前为止 也只是把Node 对象转换成了TreeNode 对象，把单向链表转换成了双向链表

        if ((tab[index] = hd) != null)
```

哈尼出品 必属精品

```
            hd.treeify(tab);  //转为红黑树
    }
}
```

为什么必须要 2 的幂次方？

1.首先，我们必须保证 table 的长度不能超过我指定的长度，所以大家想到的办法是%,取模，比如取模 16，那么我能得到 0-15 的值

2.除了取模，更好的就是位运算，那么我们能不能得到一个获取 hash 的二进制的最后几位！！假如 0-15，我能不能获取 0000-1111 的二进制！！

所以，hashMap 里面就用了(n - 1) & hash， 只有 n 为 2 的幂次方，n-1 才能都是 1111！！才能获取获取 0000-1111 的二进制！！

3.为了减少 hash 冲突，hashMap 用高低位都参与运算！！来减少冲突

其他字段都为默认值，那么我们看下默认值

```
static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // aka 16  默认大小

/**
 * The maximum capacity, used if a higher value is implicitly specified
 * by either of the constructors with arguments.
 * MUST be a power of two <= 1<<30.
 */
static final int MAXIMUM_CAPACITY = 1 << 30;   //最大容量

/**
 * The load factor used when none specified in constructor.
 */
static final float DEFAULT_LOAD_FACTOR = 0.75f;    //扩容因子

/**
 * The bin count threshold for using a tree rather than list for a
 * bin.  Bins are converted to trees when adding an element to a
 * bin with at least this many nodes. The value must be greater
 * than 2 and should be at least 8 to mesh with assumptions in
 * tree removal about conversion back to plain bins upon
 * shrinkage.
 */
static final int TREEIFY_THRESHOLD = 8;   //链表转树的临界值

/**
 * The bin count threshold for untreeifying a (split) bin during a
```

咕泡出品 必属精品

```
 * resize operation. Should be less than TREEIFY_THRESHOLD, and at
 * most 6 to mesh with shrinkage detection under removal.
 */
static final int UNTREEIFY_THRESHOLD = 6;  //树转链表的临界值

/**
 * The smallest table capacity for which bins may be treeified.
 * (Otherwise the table is resized if too many nodes in a bin.)
 * Should be at least 4 * TREEIFY_THRESHOLD to avoid conflicts
 * between resizing and treeification thresholds.
 */
static final int MIN_TREEIFY_CAPACITY = 64;    //链表转树的数组最大值
```

1.6CHM

1.6.1put 方法

1.6.1.1 第一阶段（初始化）

```
private final Node<K,V>[] initTable() {
    Node<K,V>[] tab; int sc;
    while ((tab = table) == null || tab.length == 0) {//当多个线程进来初始化的时候
        if ((sc = sizeCtl) < 0)
            Thread.yield(); // lost initialization race; just spin
        else if (U.compareAndSwapInt(this, SIZECTL, sc, -1)) {//（原子性操作，只有一个线程可以进来，并且将sizeCtl 值设置为-1，其他的线程继续循环，并且因为sizeCTL 小于0 而自动放弃时间片。
            try {
                if ((tab = table) == null || tab.length == 0) {
                    int n = (sc > 0) ? sc : DEFAULT_CAPACITY;
                    @SuppressWarnings("unchecked")
                    Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n];
                    table = tab = nt;
                    sc = n - (n >>> 2);//n-n*(0.25)=n*(1-0.25)
                }
            } finally {
                sizeCtl = sc;
            }
            break;
        }
    }
    return tab;
}
```

1.6.1.2 第二阶段（hash 冲突）

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
```

咕泡出品 必屬精品

```
//不能为空
    int hash = spread(key.hashCode());  //相对于hashMap，只能是非负数
    //HASH_BITS:01111111 11111111 11111111 11111111
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {  //自旋
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)  //table 没有数据
            tab = initTable(); //初始化
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {  //如
果定位到的槽上没有节点，则尝试通过cas 将该key value 的节点赋值到槽上
            //将引用值存储到给定的Java 变量中。这里实际上是设置一个Java 对象
o 中偏移地址为offset 的属性的值为x，此方法可以突破修饰符的抑制，也就是无视pr
ivate、protected 和default 修饰符。类似的方法有putInt、putDouble 等等。强制
从主存中获取属性值
            if (casTabAt(tab, i, null,
                         new Node<K,V>(hash, key, value, null)))      /
/通过cas 添加到节点上，保证只有1 个线程执行成功
                break;                   // no lock when adding to empt
y bin
        }
        else if ((fh = f.hash) == MOVED)  //如果对应槽位不为空，且他的 h
ash 值是 -1，说明正在扩容，那么就帮助其扩容。以加快速度
            //协助扩容
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            //通过锁，锁定哈希槽的第一个元素
            synchronized (f) {
                if (tabAt(tab, i) == f) {
                    if (fh >= 0) {
                        binCount = 1;
                        for (Node<K,V> e = f;; ++binCount) {
                            K ek;
                            if (e.hash == hash &&
                                ((ek = e.key) == key ||
                                 (ek != null && key.equals(ek)))) {
                                oldVal = e.val;
                                if (!onlyIfAbsent)
                                    e.val = value;
                                break;
                            }
                            Node<K,V> pred = e;
                            if ((e = e.next) == null) {
                                pred.next = new Node<K,V>(hash, key,
                                                          value, null);
                                break;
                            }
                        }
```

咕泡出品 必属精品

```
                    }
                    else if (f instanceof TreeBin) {
                        Node<K,V> p;
                        binCount = 2;
                        if ((p = ((TreeBin<K,V>)f).putTreeVal(hash, key,
                                                              value)) != null)
 {
                            oldVal = p.val;
                            if (!onlyIfAbsent)
                                p.val = value;
                        }
                    }
                }
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD)
                    treeifyBin(tab, i);
                if (oldVal != null)
                    return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);//增加元素个数
    return null;
}
```

1.6.1.3 第三阶段（元素个数统计和更新）（选讲）

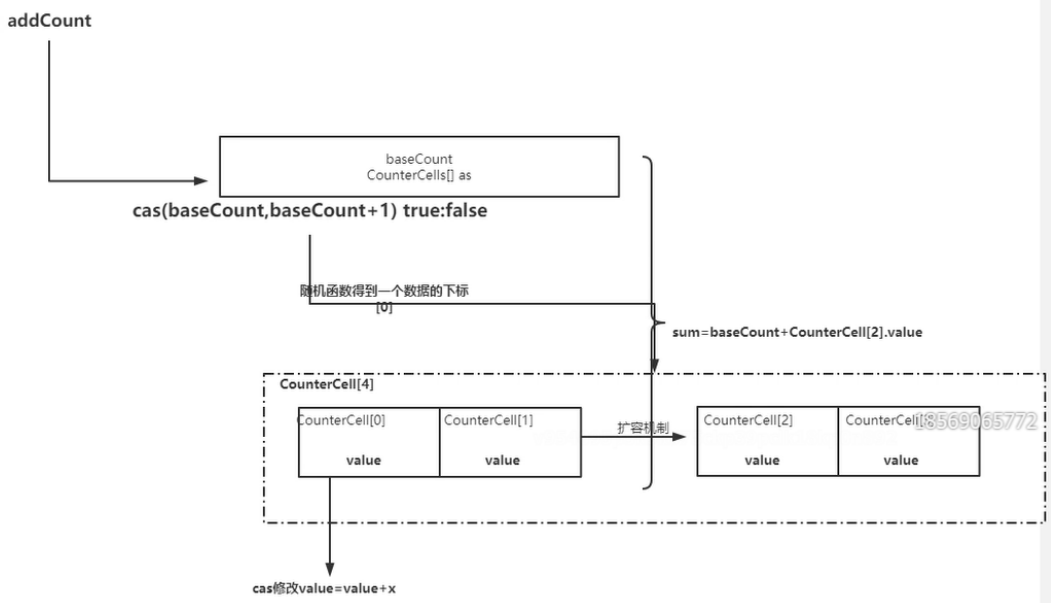
如果去更新元素的话，是不是线程并发的更新，并发更新一个共享变量的值，如何保证性能和安全性？

AddCount()添加元素个数（初始化阶段）

```
private transient volatile long baseCount;//在没有竞争的情况下，去通过 CAS 操作更新元素个数
private transient volatile CounterCell[] counterCells;//在存在线程竞争的情况下，存储元素个数
```

```
size()
      sum=baseCount+(遍历 CounterCell[]数组中每个 CounterCell 中的 value);
```

咸鱼出品 必属精品




```
// 从 putVal 传入的参数是 1， binCount，binCount 默认是0，只有 hash 冲突了才会大于 1.且他的大小是链表的长度（如果不是红黑数结构的话）。
private final void addCount(long x, int check) {
    CounterCell[] as; long b, s;
    // 如果计数盒子不是空 或者
    // 如果修改 baseCount 失败
    if ((as = counterCells) != null ||
        !U.compareAndSwapLong(this, BASECOUNT, b = baseCount, s = b +
x)) {
        CounterCell a; long v; int m;
        boolean uncontended = true;
        // 如果计数盒子是空（尚未出现并发）
        // 如果随机取余一个数组位置为空 或者
        // 修改这个槽位的变量失败（出现并发了）
        // 执行 fullAddCount 方法。并结束
        if (as == null || (m = as.length - 1) < 0 ||
            (a = as[ThreadLocalRandom.getProbe() & m]) == null ||
            !(uncontended =
              U.compareAndSwapLong(a, CELLVALUE, v = a.value, v + x)))
{
            fullAddCount(x, uncontended);
            return;
        }
        if (check <= 1)
            return;
        s = sumCount();
    }
```

```
    // 如果需要检查,检查是否需要扩容，在 putVal 方法调用时，默认就是要检查的。
    if (check >= 0) {
        Node<K,V>[] tab, nt; int n, sc;
        // 如果map.size() 大于 sizeCtl（达到扩容阈值需要扩容） 且
        // table 不是空；且 table 的长度小于 1 << 30。（可以扩容）
        while (s >= (long)(sc = sizeCtl) && (tab = table) != null &&
               (n = tab.length) < MAXIMUM_CAPACITY) {
            // 根据 length 得到一个标识
            int rs = resizeStamp(n);
            // 如果正在扩容
            if (sc < 0) {
                // 如果 sc 的低 16 位不等于 标识符（校验异常 sizeCtl 变化
了）
                // 如果 sc == 标识符 + 1 （扩容结束了，不再有线程进行扩容）
（默认第一个线程设置 sc ==rs 左移 16 位 + 2，当第一个线程结束扩容了，就会将
sc 减一。这个时候，sc 就等于 rs + 1）
                // 如果 sc == 标识符 + 65535（帮助线程数已经达到最大）
                // 如果 nextTable == null（结束扩容了）
                // 如果 transferIndex <= 0 (转移状态变化了)
                // 结束循环
                if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs + 1 |
|
                    sc == rs + MAX_RESIZERS || (nt = nextTable) == null
 ||
                    transferIndex <= 0)
                    break;
                // 如果可以帮助扩容，那么将 sc 加 1. 表示多了一个线程在帮助
扩容
                if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1))
                    // 扩容
                    transfer(tab, nt);
            }
            // 如果不在扩容，将 sc 更新：标识符左移 16 位 然后 + 2. 也就是
变成一个负数。高 16 位是标识符，低 16 位初始是 2.
            else if (U.compareAndSwapInt(this, SIZECTL, sc,
                                         (rs << RESIZE_STAMP_SHIFT) +
2))
                // 更新 sizeCtl 为负数后，开始扩容。
                transfer(tab, null);
            s = sumCount();
        }
    }
}
```

总结一下该方法的逻辑：

x 参数表示的此次需要对表中元素的个数加几。check 参数表示是否需要进行扩容检查，

咕泡出品 必属精品

大于等于 0 需要进行检查，而我们的 putVal 方法的 binCount 参数最小也是 0 ，因此，每次添加元素都会进行检查。（除非是覆盖操作）

1、判断计数盒子属性是否是空，如果是空，就尝试修改 baseCount 变量，对该变量进行加 X。
2、如果计数盒子不是空，或者修改 baseCount 变量失败了，则放弃对 baseCount 进行操作。
3、如果计数盒子是 **null** 或者计数盒子的 length 是 0，或者随机取一个位置取于数组长度是 null，那么就对刚刚的元素进行 CAS 赋值。
4、如果赋值失败，或者满足上面的条件，则调用 fullAddCount 方法重新死循环插入。
5、这里如果操作 baseCount 失败了（或者计数盒子不是 Null），且对计数盒子赋值成功，那么就检查 check 变量，如果该变量小于等于 1. 直接结束。否则，计算一下 count 变量。
6、如果 check 大于等于 0 ，说明需要对是否扩容进行检查。
7、如果 map 的 size 大于 sizeCtl（扩容阈值），且 table 的长度小于 1 << 30，那么就进行扩容。
根据 length 得到一个标识符，然后，判断 sizeCtl 状态，如果小于 0 ，说明要么在初始化，要么在扩容。
8、如果正在扩容，那么就校验一下数据是否变化了（具体可以看上面代码的注释）。如果检验数据不通过，break。
9、如果校验数据通过了，那么将 sizeCtl 加一，表示多了一个线程帮助扩容。然后进行扩容。
10、如果没有在扩容，但是需要扩容。那么就将 sizeCtl 更新，赋值为标识符左移 16 位 —— 一个负数。然后加 2。 表示，已经有一个线程开始扩容了。然后进行扩容。然后再次更新 count，看看是否还需要扩容。

```
private final void addCount(long x, int check) {
    CounterCell[] as; long b, s;
    if ((as = counterCells) != null ||
        !U.compareAndSwapLong( o: this, BASECOUNT, b = baseCount, s = b + x)) {
        CounterCell a; long v; int m;
        boolean uncontended = true;
        if (as == null || (m = as.length - 1) < 0 |
            (a = as[ThreadLocalRandom.getProbe() & m]) == null ||
            !(uncontended =
              U.compareAndSwapLong(a, CELLVALUE, v = a.value, l2: v + x))) {
            fullAddCount(x, uncontended);
            return;
        }
        if (check <= 1)
            return;
        s = sumCount();
    }
```

baseCousnt+x

1)
直接去访问 baseCount 累加元素个数

2）找到 CounterCell[]随机的某个下标位置，value=v+x->表示记录元素个数

3）如果前面都失败，则进入到 fullAddCount

CounterCell[]为 null

```
else if (cellsBusy == 0 && counterCells == as &&
        U.compareAndSwapInt( o: this, CELLSBUSY, i: 0, i1: 1)) {
    boolean init = false;
    try {                           // Initialize table
        if (counterCells == as) {
            CounterCell[] rs = new CounterCell[2];
            rs[h & 1] = new CounterCell(x);
            counterCells = rs;
            init = true;
        }
    } finally {
        cellsBusy = 0;
    }
```

元素个数的值

表示当前线程获得了初始化的资格

已经初始化了，然后存在竞争，CAS 进行更新

如果 CAS 失败，触发 CounterCell 扩容

AddCount()添加元素个数（元素个数更新的阶段）

1.6.1.4 第四阶段（协助扩容选讲）

- 当元素个数大于阈值的时候触发扩容
- 如果此时正在扩容，在扩容阶段进来的线程会协助扩容

```
static final int resizeStamp(int n) {
    return Integer.numberOfLeadingZeros(n) | (1 << (RESIZE_STAMP_BITS -
 1));
    //Integer.numberOfLeadingZeros(n)这个方法是用来取高位0的个数
}
```

在 putVal 方法中，如果发现线程当前 hash 冲突了，也就是当前 hash 值对应的槽位有值了，且如果这个值是 -1 （MOVED），说明 Map 正在扩容。那么就帮助 Map 进行扩容。以加快速度。

```
final Node<K,V>[] helpTransfer(Node<K,V>[] tab, Node<K,V> f) {
    Node<K,V>[] nextTab; int sc;
```

```
    // 如果 table 不是空 且node 节点是转移类型，数据检验
    // 且 node 节点的 nextTable（新 table） 不是空，同样也是数据校验
    // 尝试帮助扩容
    if (tab != null && (f instanceof ForwardingNode) &&
        (nextTab = ((ForwardingNode<K,V>)f).nextTable) != null) {
        // 根据 length 得到一个标识符号
        int rs = resizeStamp(tab.length);
        // 如果 nextTab 没有被并发修改 且 tab 也没有被并发修改
        // 且 sizeCtl  < 0 （说明还在扩容）
        while (nextTab == nextTable && table == tab &&
               (sc = sizeCtl) < 0) {
            // 如果 sizeCtl 无符号右移  16 不等于 rs （ sc 前 16 位如果不等于标识符，则标识符变化了）
            // 或者 sizeCtl == rs + 1  （扩容结束了，不再有线程进行扩容）（默认第一个线程设置 sc ==rs 左移 16 位 + 2，当第一个线程结束扩容了，就会将 sc 减一。这个时候，sc 就等于 rs + 1）
            // 或者 sizeCtl == rs + 65535  （如果达到最大帮助线程的数量，即 65535）
            // 或者转移下标正在调整 （扩容结束）
            // 结束循环，返回 table
            if ((sc >>> RESIZE_STAMP_SHIFT) != rs || sc == rs + 1 ||
                sc == rs + MAX_RESIZERS || transferIndex <= 0)
                break;
            // 如果以上都不是，将 sizeCtl + 1，（表示增加了一个线程帮助其扩容）
            if (U.compareAndSwapInt(this, SIZECTL, sc, sc + 1)) {
                // 进行转移
                transfer(tab, nextTab);
                // 结束循环
                break;
            }
        }
        return nextTab;
    }
    return table;
}
```

关于 sizeCtl 变量：

-1 :代表 table 正在初始化,其他线程应该交出 CPU 时间片
-N: 表示正有 N-1 个线程执行扩容操作（高 16 位是 length 生成的标识符，低 16 位是扩容的线程数）
大于 0: 如果 table 已经初始化,代表 table 容量,默认为 table 大小的 0.75,如果还未初始化,代表需要初始化的大小

代码步骤：

咕泡出品 必属精品

1. 判 tab 空，判断是否是转移节点。判断 nextTable 是否更改了。
2. 更加 length 得到标识符。
3. 判断是否并发修改了，判断是否还在扩容。
4. 如果还在扩容，判断标识符是否变化，判断扩容是否结束，判断是否达到最大线程数，判断扩容转移下标是否在调整（扩容结束），如果满足任意条件，结束循环。
5. 如果不满足，并发转移。

这里有一个花费了很长时间纠结的地方：

```
sc == rs + 1
```

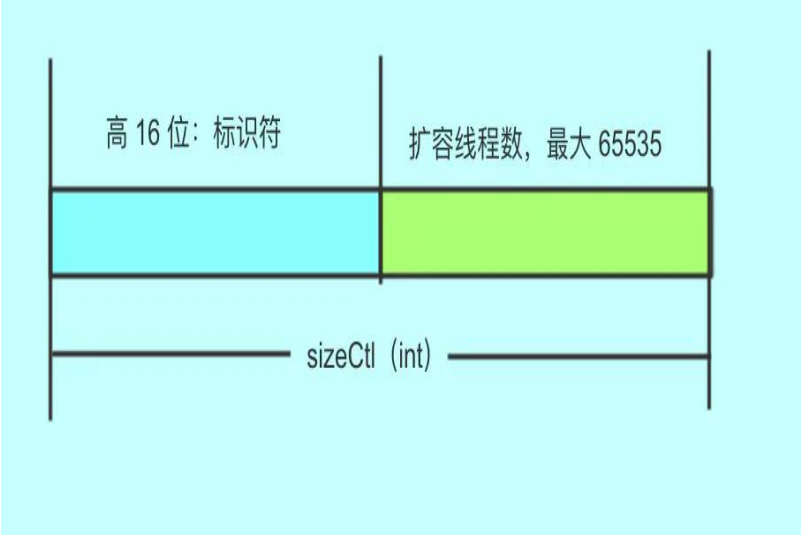
这个判断可以在 addCount 方法中找到答案：默认第一个线程设置 sc ==rs 左移 16 位 + 2，当第一个线程结束扩容了，就会将 sc 减一。这个时候，sc 就等于 rs + 1。

如果 sizeCtl == 标识符 + 1 ，说明库容结束了，没有必要再扩容了。

总结一下：

当 Cmap put 元素的时候，如果发现这个节点的元素类型是 forward 的话，就帮助正在扩容的线程一起扩容，提高速度。其中， sizeCtl 是关键，该变量高 16 位保存 length 生成的标识符，低 16 位保存并发扩容的线程数，通过这连个数字，可以判断出，是否结束扩容了。

如下图：



1.7 面试题

1.7.1 HashMap 的数据结构？

JDK7：数组+链表

JDK8: 数组+链表+红黑树（看过源码的同学应该知道 JDK8 中即使用了单向链表，也使用了双向链表，双向链表主要是为了链表操作方便，应该在插入，扩容，链表转红黑树，红黑树转链表的过程中都要操作链表）

数组+链表（哈希表）；当链表长度超过 8 且数组长度超过 64，链表转换位红黑树，变成数组+红黑树，当红黑树长度小于 6 则转成链表

1.7.2 JDK8 中的 HashMap 为什么要使用红黑树？

当元素个数小于一个阈值时，链表整体的插入查询效率要高于红黑树，当元素个数大于此阈值时，链表整体的插入查询效率要低于红黑树。此阈值在 HashMap 中为 8

1.7.3 HashMap 中的 hash 是怎么实现的？有什么好处？

是通过 hashCode()的高 16 位异或低 16 位实现的:`(h=k.hashCode())^(h>>>16)`，让 hash 的高低位都参与 hash 运算，减少 hash 碰撞！！！

1.7.3 jdk8 中对 HashMap 做了哪些改变?

1）JDK8 中使用了红黑树
2）JDK7 中链表的插入使用的头插法（扩容转移元素的时候也是使用的头插法，头插法速度更快，无需遍历链表，但是在多线程扩容的情况下使用头插法会出现循环链表的问题，导致 CPU 飙升），JDK8 中链表使用的尾插法（JDK8 中反正要去计算链表当前结点的个数，反正要遍历的链表的，所以直接使用尾插法）
3）JDK7 的 Hash 算法比 JDK8 中的更复杂，Hash 算法越复杂，生成的 hashcode 则更散列，那么 hashmap 中的元素则更散列，更散列则 hashmap 的查询性能更好，JDK7 中没有红黑树，所以只能优化 Hash 算法使得元素更散列，而 JDK8 中增加了红黑树，查询性能得到了保障，所以可以简化一下 Hash 算法，毕竟 Hash 算法越复杂就越消耗 CPU
4）扩容的过程中 JDK7 中有可能会重新对 key 进行哈希（重新 Hash 跟哈希种子有关系），而 JDK8 中没有这部分逻辑
5）JDK8 中扩容的条件和 JDK7 中不一样，除开判断 size 是否大于阈值之外，JDK7 中还判断了 tab[i]是否为空，不为空的时候才会进行扩容，而 JDK8 中则没有该条件了
6）JDK8 中还多了一个 API：putIfAbsent(key,value)
JDK7 和 JDK8 扩容过程中转移元素的逻辑不一样，JDK7 是每次转移一个元素，

昆明出品 必属精品

JDK8 是先算出来当前位置上哪些元素在新数组的低位上，哪些在新数组的高位上，然后在一次性转移

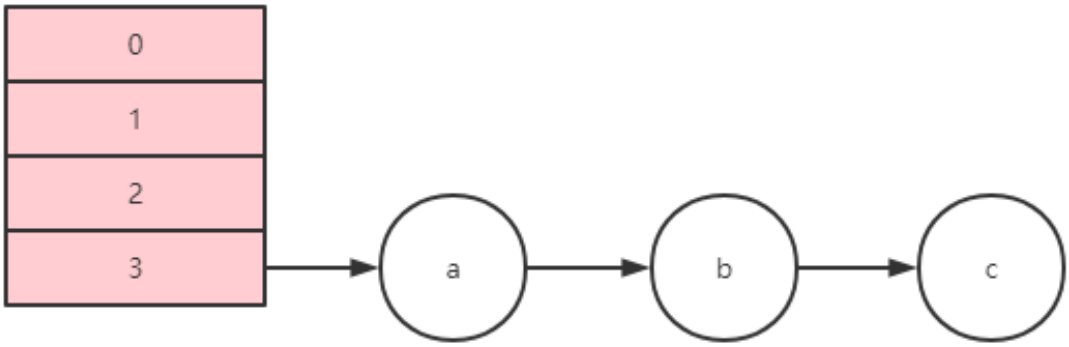
1.7.4 HashMap 为什么不是线程安全的

多线程环境下，会发生数据覆盖的情况

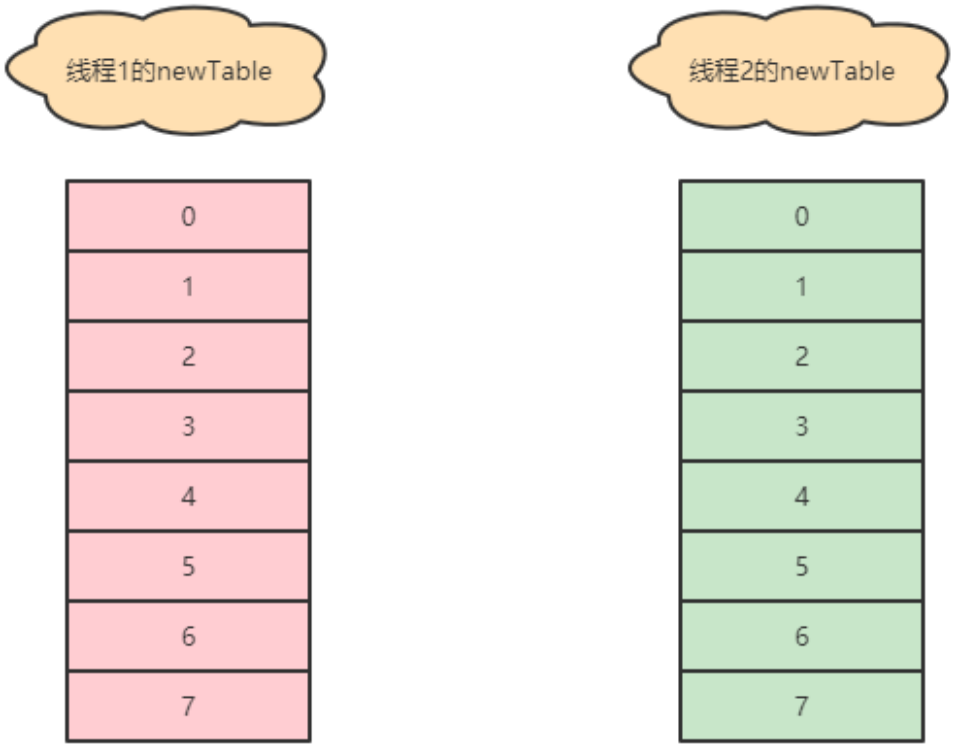
resize 的时候会发生死循环

1.7HM

```
void transfer(Entry[] newTable, boolean rehash) {
    int newCapacity = newTable.length;
    for (Entry<K,V> e : table) {
        while(null != e) {
            Entry<K,V> next = e.next;
            if (rehash) {
                e.hash = null == e.key ? 0 : hash(e.key);
            }
            int i = indexFor(e.hash, newCapacity);
            e.next = newTable[i];
            newTable[i] = e;
            e = next;
        }
    }
}
```

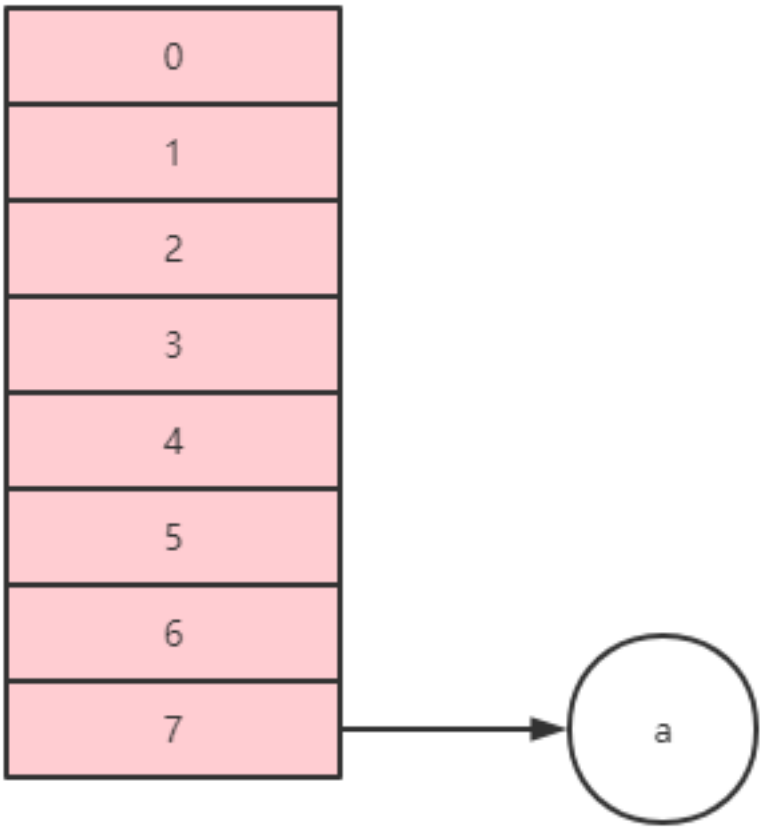


插入第 4 个节点时，发生 rehash，假设现在有两个线程同时进行，线程 1 和线程 2，两个线程都会新建新的数组

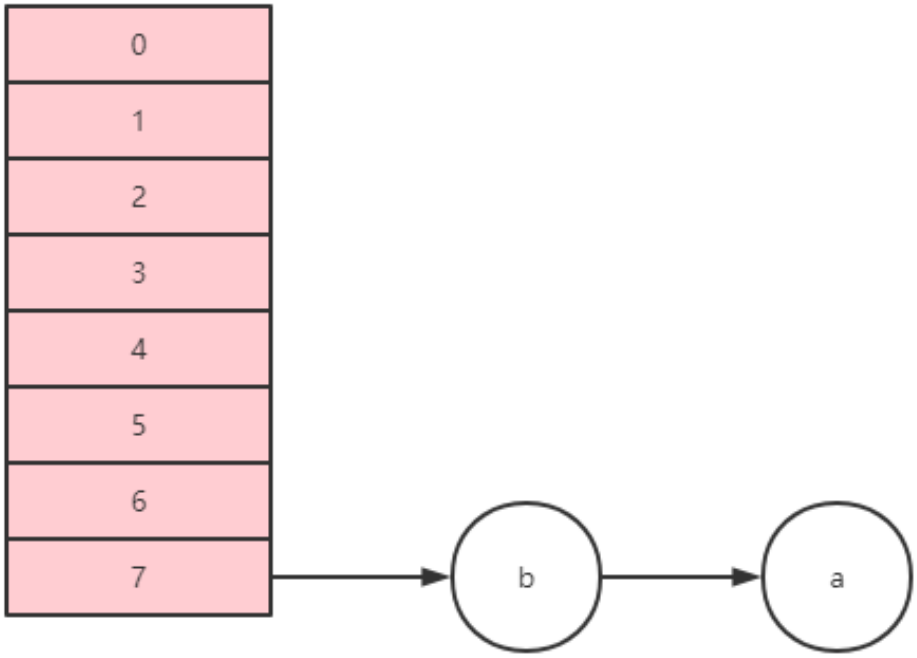


假设线程 2 在执行到 Entry < K,V > next = e.next;之后，cpu 时间片用完了，这时变量 e 指向节点 a，变量 next 指向节点 b。
线程 1 继续执行，很不巧，a、b、c 节点 rehash 之后又是在同一个位置 7，开始移动节点
第一步，移动节点 a

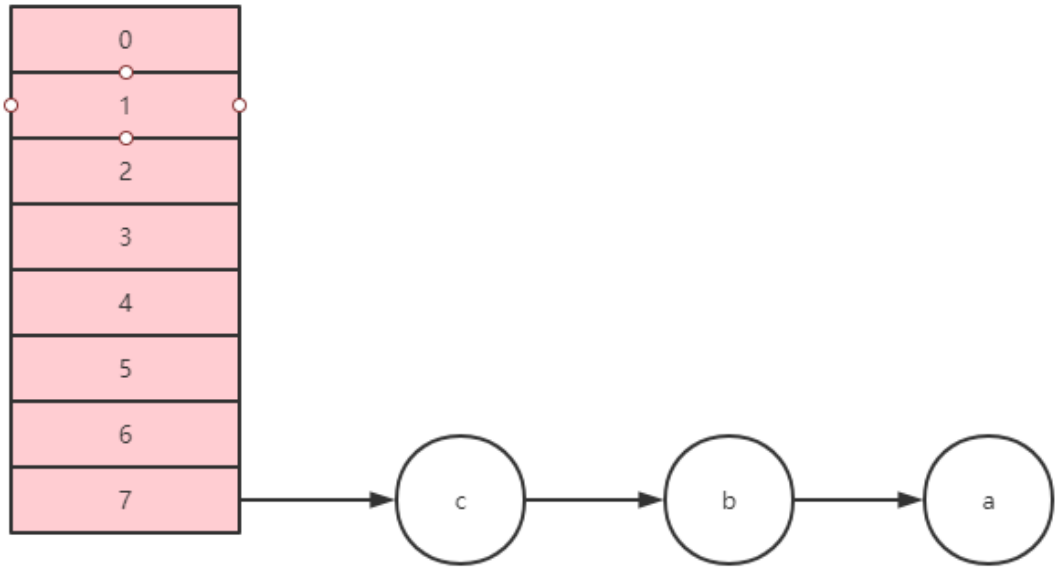
咕泡出品



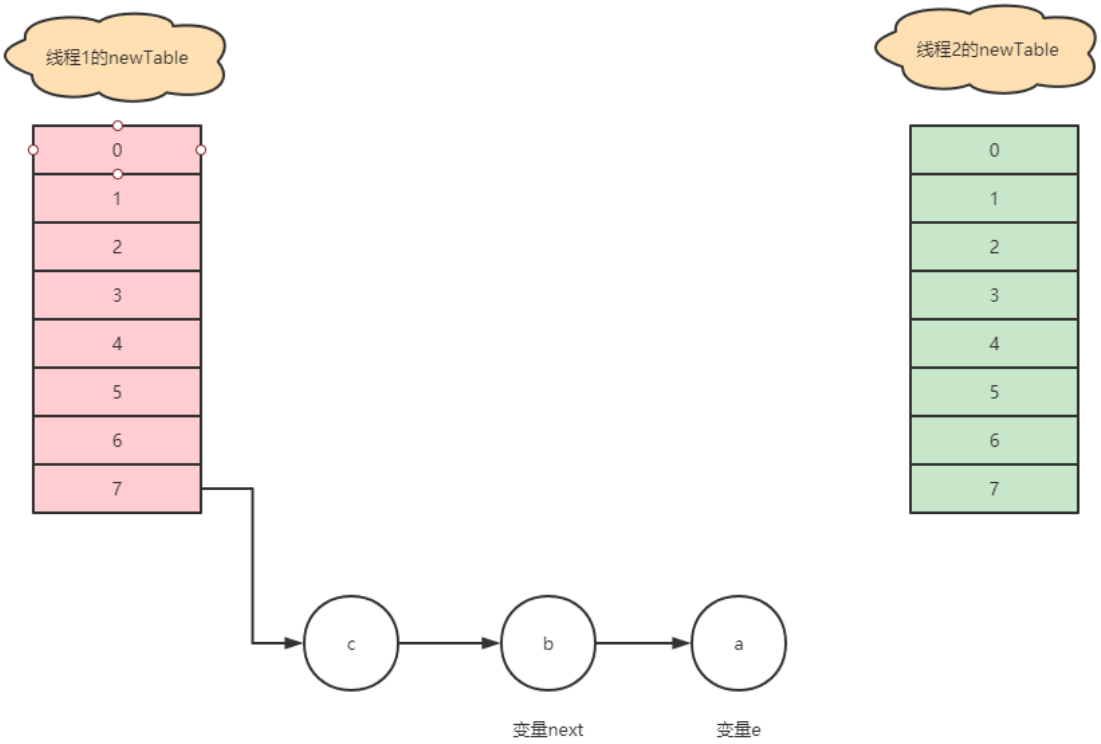
第二步，移动节点 b



注意，这里的顺序是反过来的，继续移动节点 c



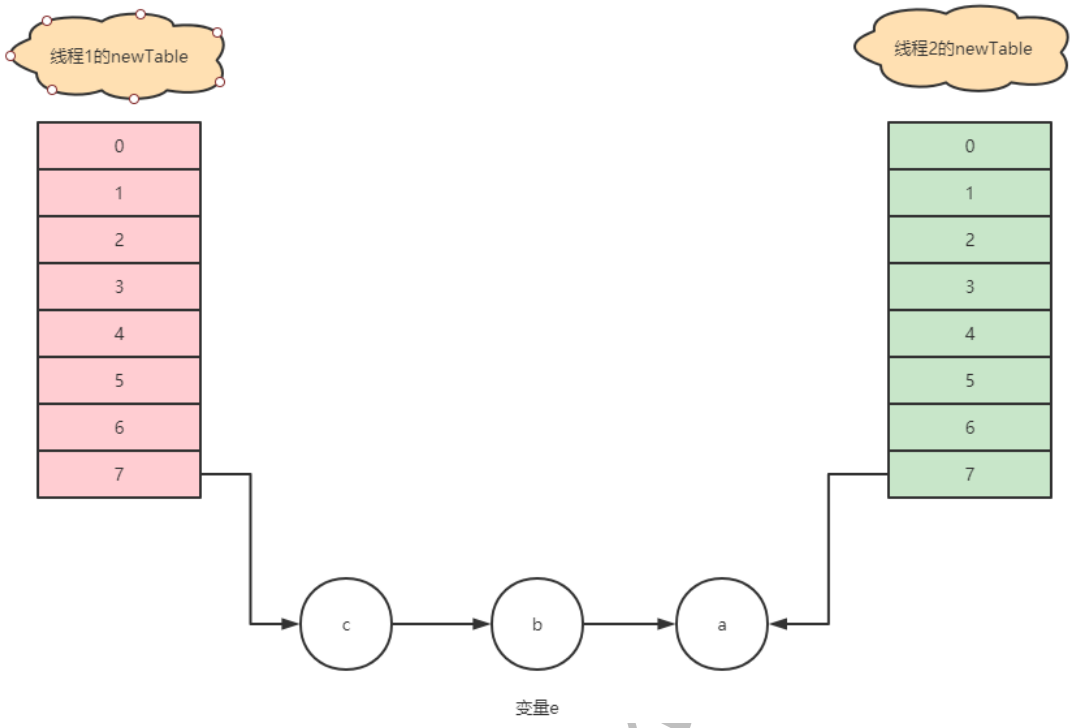
这个时候 线程 1 的时间片用完，内部的 table 还没有设置成新的 newTable， 线程 2 开始执行，这时内部的引用关系如下：



这时，在 线程 2 中，变量 e 指向节点 a，变量 next 指向节点 b，开始执行循环体的剩余逻辑。

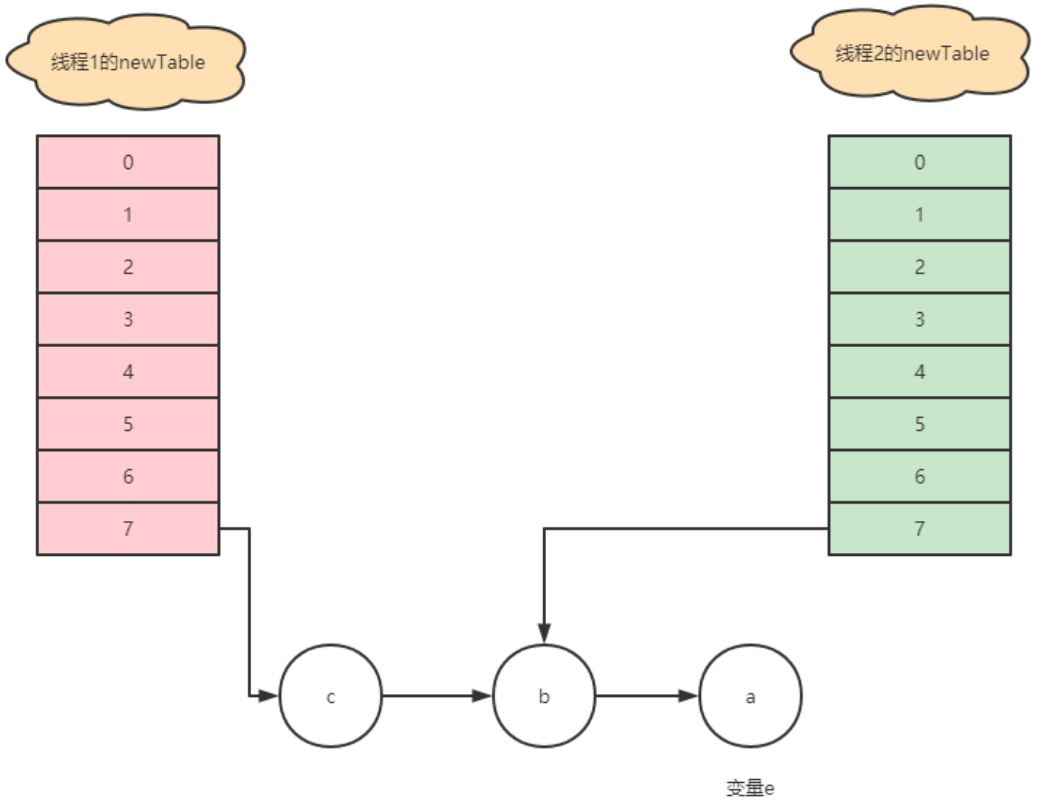
```
Entry<K,V> next = e.next;
int i = indexFor(e.hash, newCapacity);
e.next = newTable[i];
newTable[i] = e;
e = next;
```

执行之后的引用关系如下图



咕泡出品必

执行后，变量 e 指向节点 b，因为 e 不是 null，则继续执行循环体，执行后的引用关系



变量 e 又重新指回节点 a，只能继续执行循环体，这里仔细分析下：

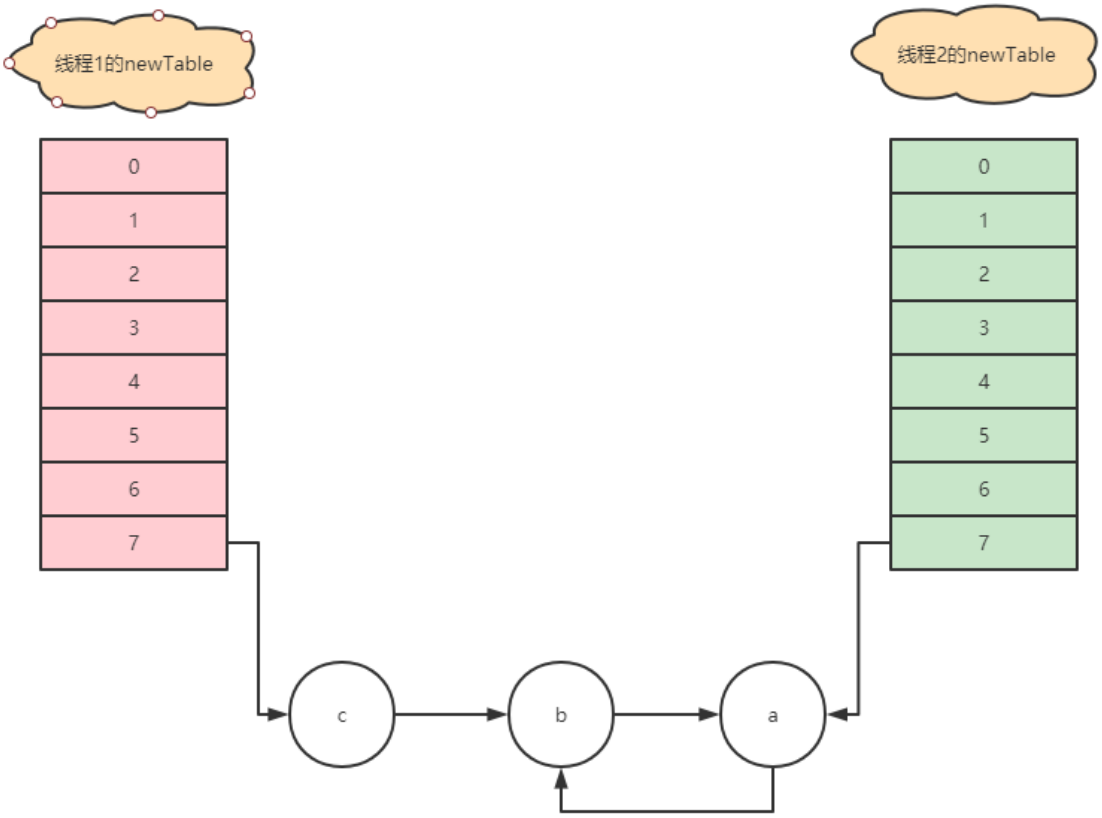
1、执行完 Entry < K,V > next = e.next;，目前节点 a 没有 next，所以变量 next 指向 null；

2、e.next = newTable[i]; 其中 newTable[i] 指向节点 b，那就是把 a 的 next 指向了节点 b，这样 a 和 b 就相互引用了，形成了一个环；

3、newTable[i] = e 把节点 a 放到了数组 i 位置；

4、e = next; 把变量 e 赋值为 null，因为第一步中变量 next 就是指向 null；

所以最终的引用关系是这样的：



节点 a 和 b 互相引用，形成了一个环，当在数组该位置 get 寻找对应的 key 时，就发生了死循环。

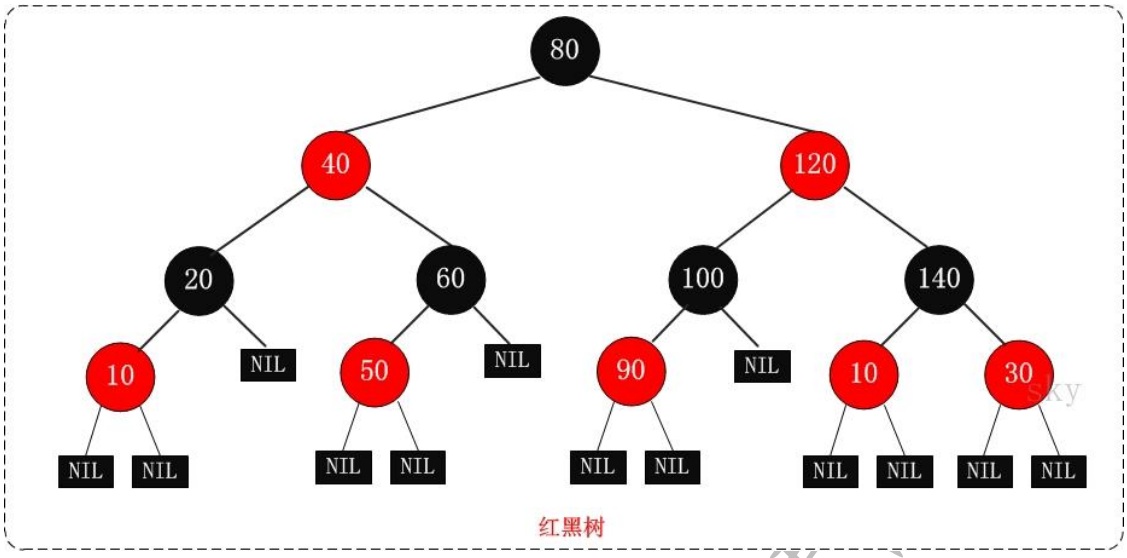
另外，如果线程 2 把 newTable 设置成到内部的 table，节点 c 的数据就丢了，看来还有数据遗失的问题。

总结
所以在并发的情况，发生扩容时，可能会产生循环链表，在执行 get 的时候，会触发死循环，引起 CPU 的 100%问题，所以一定要避免在并发环境下使用 HashMap。

咕泡出品

1.7.5 说说你对红黑树的见解?

- 每个节点非红即黑
- 根节点总是黑色的
- 如果节点是红色的,则它的子节点必须是黑色的(反之不一定)
- 每个叶子节点都是黑色的空节点(NIL 节点)
- 从根节点到叶节点或空子节点的每条路径,必须包含相同数目的黑色节点(即相同的黑色高度)



-
-

1.7.6 hashMap 的容量怎么扩充

- table 数组大小是由 capacity 这个参数确定的，默认是 16，也可以构造时传入，最大限制是 1<<30；并且必须是 2 的幂次方
- loadFactor 是装载因子，主要目的是用来确认 table 数组是否需要动态扩展，默认值是 0.75，比如 table 数组大小为 16，装载因子为 0.75 时，threshold 就是 12，当 table 的实际大小超过 12 时，table 就需要动态扩容；
- 扩容时，调用 resize() 方法，将 table 长度变为原来的两倍，并且根据新的容量，计算新的数据索引值

1.7.7 hashMap 为什么在单链表达到 8 后要用红黑树，不用二叉查找树，为什么节点小于 6 的时候又要转变为链表

二叉查找树有斜树的情况，相当于链表！而红黑树会有旋转、变色保证平衡！查找快，但是插入修改会相对较慢！

所以，当节点小的时候，为什么要转为链表！

1.7.8 hashMap 与 chm 的区别

加锁跟没加锁的区别。chm 不允许 null 的 key 跟 value!hashMap 允许。

1.7.9 hashMap 在 JDK1.8 跟 1.7 的区别，chm 在 JDK1.8 跟 1.7 的区别

hashMap

- 1.7 采用数组+单链表，1.8 在单链表超过 8 并且数组长度超过 64 时，单链表更改改成红黑树存储，
- 1.7 扩容时需要重新计算哈希值和索引位置，1.8 不重新计算哈希值，而是采用和扩容后容量进行&操作来计算新的索引位置。
- 1.7 插入元素到单链表中采用头插入法，1.8 采用的是尾插入法。（1.7 重新计算 hash 值并发有闭环链表风险）

chm 上述 hashMap 的变更也是 chm 的变更，还有锁

1.7 采用分段锁的机制，锁住的是一个 segment 数组，采用重入锁 ReentrantLock（重入锁）

1.8.采用 Node + CAS + Synchronized 来保证并发安全,取消类 Segment，直接锁 entry

为什么取消重入锁，采用 synchronized?

- 粒度降低了；
- JVM 开发团队没有放弃 synchronized，而且基于 JVM 的 synchronized 优化空间更大，更加自然。
- 在大量的数据操作下，对于 JVM 的内存压力，基于 API 的 ReentrantLock 会开销更多的内存。

咕泡出品 必属精品

1.7.10 put 跟 putIfAbsent 的区别

put 覆盖原有的值，putIfAbsent 不覆盖原有值

1.7.11 HashMap 的数据结构

在 1.7 之前，数组+单链表

1.8，数组+链表 或者数组+红黑树,链表超过 8 并且数组长度大于 64，转红黑树，如果树节点小于 6，转为链表

咕泡出品 必属精品